kacchiOS Project Report

CSE 3202 - Operating Systems Sessional. Reference implementation of the Memory Manager, Process Manager and Scheduler on the kacchiOS bare-metal starter.

1. Introduction

kacchiOS is a tiny 32-bit x86 operating system that boots through a Multiboot loader and talks to the outside world over the serial port. The starter already contained the boot code, a serial driver and a Null Process that reads one line of input and echoes it back. This project adds the three components of the assignment: a memory manager, a process manager and a scheduler with a real context switch.

2. Objectives

- Allocate and free memory for processes (stacks and heap), with reuse after free.
- Manage a process table with creation, state transitions and termination.
- Switch between processes with the hardware timer and a documented policy.
- Keep the code simple enough to explain in one evening at a viva.

3. Existing system architecture

Boot flow after QEMU loads the kernel at 1 MB:

```
boot.S   set ESP to a 16 KB boot stack, clear .bss, call kmain()
kmain()  serial_init(); create the null process; scheduler_start()
shell    the null process (pid 0) prints 'kacchiOS>' and reads a line
```

Existing files kept unchanged in behaviour: serial.c/h (COM1 driver), string.c/h, types.h, io.h (inb/outb), link.ld (kernel layout, exports __kernel_end). The sources were moved into src/ to match the required submission structure.

4. Memory manager

Memory layout (QEMU runs with -m 64M):

```
  1 MB          kernel image (.text, .data, .bss)
  __kernel_end  heap starts here (8 MB)
  64 MB         end of RAM
```

Stacks: the stack pool is 16 fixed slots of 8 KB inside the kernel .bss (stack_pool[][]). One slot belongs to each process. allocate_stack() scans for a free slot; free_stack() validates the pointer (inside the pool? exactly at a slot base? slot actually in use?) and marks it free again, so double free and invalid free are detected and ignored with a message.

Heap: the region after the kernel is one address-ordered list of blocks with a 16-byte header (size, in_use, next, prev). allocate_heap() uses first fit and splits a block when the remainder is still useful; free_heap() marks the block free and merges it with free neighbours, so the heap returns to a single large block after everything is freed. Out of memory returns NULL.

5. Process manager

The process table is an array of 16 entries (like Xinu's proctab[NPROC]); the pid is simply the index. Each entry (PCB) holds: in_use, pid, state, name, entry function, stack_base, stack_size, saved_esp, priority, wait_ticks and a one-slot mailbox.

process_create() finds a free entry, asks the memory manager for a stack, builds the initial stack frame and only then makes the entry visible (in_use = 1), so a timer interrupt in the middle of creation cannot see a half-built process. process_set_state() is the only function that changes a state and rejects illegal transitions. When a process finishes, process_exit() marks it TERMINATED and yields; the scheduler frees its stack at the next scheduling point, but never from the stack that is still in use.

6. Process states and transitions

```
   create            scheduler picks it
(free) --> READY -------------------------> CURRENT
             ^         ^                       |
             |         | ipc_send wakes        | scheduler rotates
```

```
            |         |                      v
            +---------+                    READY
          BLOCKED <---- CURRENT
      (waits for message, only via ipc_receive_wait)
                                  finish: --> TERMINATED
                                           --> stack freed, slot reused
```

All transitions go through process_set_state(); e.g. TERMINATED -> READY is refused, so a finished process can never run again.

7. Scheduler

Policy: round robin with a configurable quantum (default 5 ticks = 50 ms at 100 Hz). Every READY process gets wait_ticks++ each tick. The next process is the READY process with the highest priority + wait_ticks/10; ties are broken in round-robin order (the scan starts after the process that ran last).

When no other process is READY, the current one simply keeps the CPU. The null process (pid 0) never terminates, so there is always at least one runnable process.

8. Context switching

A process is its stack: everything that must be preserved lives on that stack. On every timer tick:

```
timer_isr (isr.S)                 scheduler_tick (scheduler.c)
------------------                ----------------------------
pusha                 ; save 8 regs
mov %esp,%ecx
push %ecx
call scheduler_tick  ------------> save old ESP in the PCB
                                    pick next READY process
                    <-------------- return its saved ESP
mov %eax,%esp         ; switch stacks
popa                  ; restore registers
iret                  ; continue the new process
```

A new process starts with a fake frame built by build_initial_stack(): EFLAGS (interrupts on), CS, EIP = process_trampoline, plus dummy values for popa. The first iret therefore 'returns' into process_trampoline(), which calls the process entry function. yield() is just 'int $0x20' - the same interrupt path as a real timer tick.

9. Time quantum

The quantum is counted in timer ticks (100 ticks = 1 second). While the current process is CURRENT, each tick decrements the counter; at zero, the scheduler picks another READY process. The shell command 'quantum <ticks>' changes it at runtime; values are clamped to 1..100.

10. Aging

Aging needs no extra code path: the pick score already contains wait_ticks/AGING_STEP (10). A low-priority process that waits long enough outranks a high-priority process that keeps running, so nobody can starve. This is visible with the 'busy' command: the shell (priority 2) spins for N ticks while the workers (priority 1) still print their heartbeats.

11. Testing

All tests were run in QEMU 11.0 with the built kernel. Summary:

Test	How	Result
Boot + shell	make run, type text	PASS
Context switch	spawn twice, watch heartbeats between prompts	PASS
Resume after preemption	type slowly while workers print	PASS
States	ps before/after spawn, send, kill	PASS
BLOCKED + IPC	server; ps; send 1 hello	PASS
Termination + cleanup	kill 1; mem; spawn (pid reused)	PASS
Table exhaustion	spawn 16 times; last fails cleanly	PASS
Stack exhaustion	memtest: allocates until NULL	PASS
Double free / invalid free	memtest: both reported and ignored	PASS
Out of memory	memtest: huge request returns NULL	PASS

Test	How	Result
Coalescing	memtest: free bytes restored exactly	PASS
Quantum config	quantum 1 / 50 / 100 / 0 / 99999 (clamped)	PASS
Aging	busy 300: low priority workers still run	PASS
Soak test	30 s of create/terminate/quantum churn	PASS

12. Limitations

- No user mode or paging: every process is a kernel thread (Xinu model).
- Single CPU; switching relies on interrupts being off inside the ISR.
- One-slot mailbox: a second send before the first is read fails.
- Fixed 8 MB heap, sized for -m 64M in the Makefile.
- Unexpected interrupts (including CPU exceptions) halt the kernel with a message instead of being decoded.
- The scheduler decides only at quantum boundaries and at block/terminate events, not on every tick.

13. Conclusion

The three required components work together on real hardware emulation: the memory manager gives every process a stack and reuses everything that is freed, the process manager keeps a clean lifecycle with explicit state transitions, and the scheduler performs genuine context switches through the timer interrupt, with a configurable quantum and aging against starvation. The code stays small on purpose so that each function can be explained during the viva.